PHANTOM.

FINANCIAL FRAUD INTELLIGENCE SYSTEM

DBMS MINI PROJECT REPORT

A local-first transaction monitoring and fraud investigation platform built with Python, SQLite, HTML, CSS, and JavaScript.

Submitted by	_____
Course / subject	Database Management Systems
Submitted to	_____
Institution	_____
Submission date	20 October 2026

PROJECT NOTE / This report describes an educational simulation using fictional customer and transaction records. It is not connected to a bank or payment network.

01 / OVERVIEW

Abstract & project goals

Why the system exists and what the implementation demonstrates.

PHANTOM (Financial Fraud Intelligence System) is a database-driven application for monitoring transaction activity, identifying suspicious patterns, triaging alerts, and recording investigation events. Analysts can also add fictional customers, linked accounts, and transaction records through validated forms; the records are persisted in SQLite and can immediately flow through fraud scanning and audit reporting. A responsive browser dashboard communicates with a small Python HTTP API, while SQLite stores the normalized relational data. The project is designed to demonstrate core DBMS principles in a working application rather than as disconnected SQL examples.

The demo includes 18 fictional customers, 24 accounts, 94 transaction records, and 15 seeded fraud alerts. A transparent rule engine can re-evaluate transactions for large-ticket activity, high-value payments, cross-border activity, short-window transaction velocity, and high-value ATM withdrawals. Analysts can change alert status, inspect accounts, export transactions to CSV, and simulate an internal transfer. Database constraints and an atomic transaction protect account balances during that transfer.

Problem statement

Transaction monitoring involves related records, operational status changes, and a need to explain why an activity was flagged. A flat file or hard-coded dashboard does not adequately demonstrate referential integrity, queryable relationships, consistent updates, or auditability. PHANTOM models these relationships in SQLite and provides a user interface to exercise the database operations.

Objectives

- Design a relational schema for customers, accounts, transactions, fraud alerts, and audit events.
- Apply primary keys, foreign keys, uniqueness rules, check constraints, indexes, views, and triggers.
- Expose backend endpoints for dashboard metrics, searching, user-entered customer/account/transaction creation, alert decisions, CSV export, scans, and simulated transfers.
- Demonstrate atomic transfer logic and safe rejection of insufficient-funds requests.
- Present the system through a polished dark operations dashboard and package reproducible data plus submission documentation.

Scope and assumptions

All data is fictional, and transfers are local balance simulations. Fraud rules are deliberately transparent and illustrative. The prototype is intended for a classroom demonstration, not production fraud decisions, real-money movement, or use with personally identifiable financial data.

02 / SYSTEM DESIGN

Architecture & modules

A simple local architecture with no cloud service or third-party package dependency.

LAYER	IMPLEMENTATION	RESPONSIBILITY
User interface	HTML5, CSS, JavaScript, SVG	Navigation, dashboard charts, search/filter, customer/account and transaction forms, alert actions, account cards, transfer modal.
Application API	Python 3 standard library	Serves static files and JSON endpoints; validates input; applies fraud rules; manages transactions.
Persistence	SQLite + Python sqlite3	Relational storage, constraints, indexes, views, triggers, query execution, transaction boundaries.
Demo data	schema.sql + seed.sql	Reproducible fictional records that populate the database on first start or after reset.

Main application modules

MODULE	USER VALUE	DBMS CONCEPTS EXERCISED
Command center	Transaction volume, flagged exposure, active alerts, monitored account count, trend chart, risk distribution.	COUNT, SUM, GROUP BY, date aggregation, views.
Transactions	Search/filter transactions, add transaction records, and export results.	Parameterized queries, joins, inserts, indexes, audit trigger.
Fraud alert queue	Inspect rules, severity, reason, and status; investigate, resolve, reopen, or mark false positive.	Foreign keys, UNIQUE constraint, update transactions, audit inserts.
Account intelligence	Account cards and a form to create a customer with a linked opening account.	Normalized relation, atomic inserts, aggregates, SQL view.
SQL insights	Category totals, alert outcomes, query examples.	GROUP BY, AVG, SUM, views, query plans.
Audit trail	Inspect system and analyst events with actor, entity, detail, and timestamp.	Trigger-generated records and explicit transactional event logging.

Technology decisions

The project uses Python’s standard-library HTTP server and SQLite to keep installation simple and runtime local. Native SVG charts avoid external JavaScript packages and CDN access.

03 / DATA MODEL

Entity relationship design

The core tables are normalized and connected through explicit foreign-key relationships.

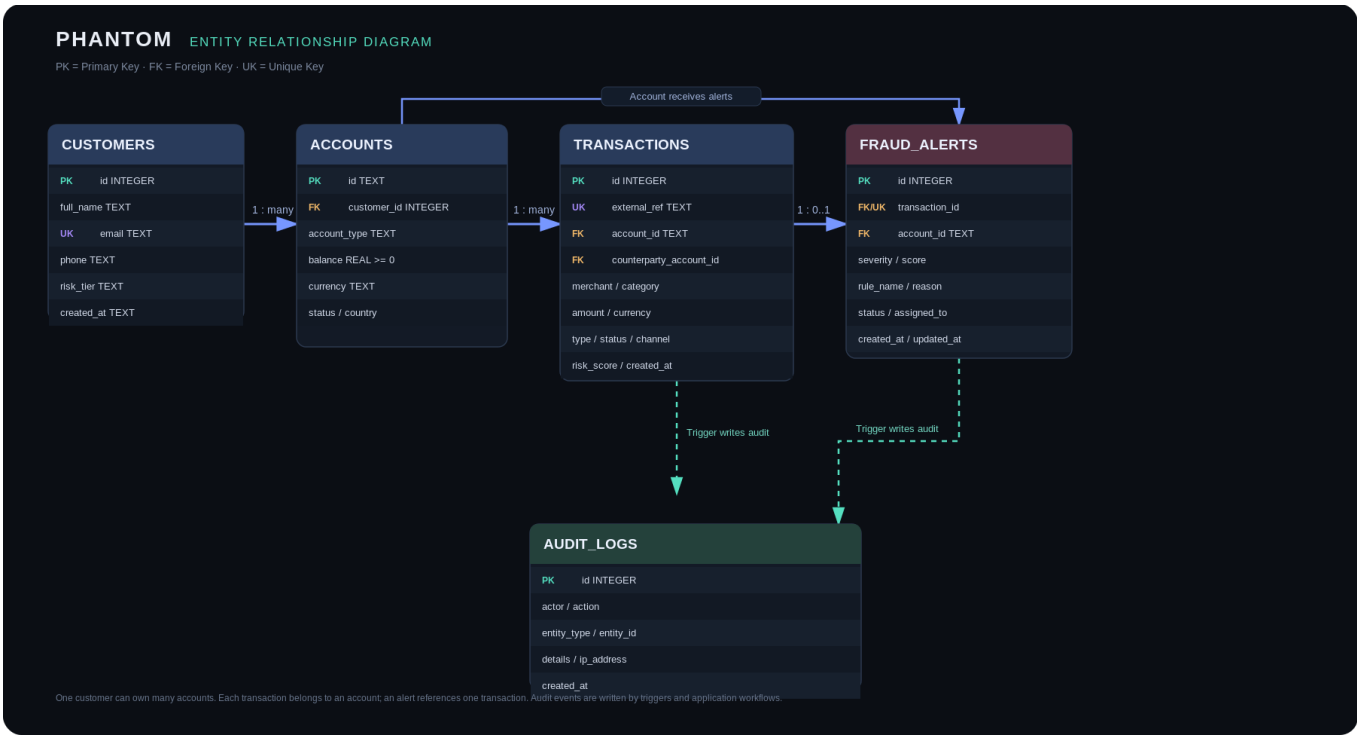


Figure 1. PHANTOM entity relationship diagram (audit logs use entity references rather than cascading foreign keys).

ENTITY	KEY FIELDS / ROLE	RELATIONSHIP
customers	id (PK), email (UNIQUE), full_name, risk_tier	One customer owns one or more accounts.
accounts	id (PK), customer_id (FK), balance, type, status	An account belongs to one customer and has a transaction history.
transactions	id (PK), external_ref (UNIQUE), account_id (FK), optional counterparty_account_id (FK), amount, risk_score	Each transaction belongs to an account and may reference a counterparty account.
fraud_alerts	id (PK), transaction_id (FK + UNIQUE), account_id (FK), severity, score, status	One transaction can have at most one alert record in this demo model.
audit_logs	id (PK), actor, action, entity_type, entity_id, details, created_at	Captures system events without being cascade-deleted with business records.

04 / DATABASE IMPLEMENTATION

Integrity, normalization & performance

Database rules do the safety work, not just the frontend.

Normalization overview

FORM	APPLICATION IN PHANTOM
First Normal Form (1NF)	Each field stores one value: customer email, account balance, transaction amount, and alert status are individual attributes, not repeating lists.
Second Normal Form (2NF)	Core business entities use single-column surrogate primary keys. Non-key attributes describe their table entity; transaction and alert details are not duplicated into the customer row.
Third Normal Form (3NF)	Customer identity details are stored in customers; account attributes depend on account ID; transaction attributes depend on transaction ID; alert disposition depends on alert ID. Customer fields are not duplicated across each transaction.

Constraints & safety properties

- Foreign keys connect accounts to customers, transactions to accounts, and alerts to both their transaction and account. Foreign-key enforcement is enabled for each connection.
- CHECK constraints limit account type/status, transaction type/status, alert severity/status, boolean international flags, risk scores (0–100), positive transaction amounts, and non-negative balances.
- Unique values prevent duplicate customer emails, transaction references, and duplicate alert rows for the same transaction.
- Parameterized SQL keeps user search input out of dynamically assembled SQL syntax.
- The transfer operation begins with BEGIN IMMEDIATE, reads and validates both accounts, debits and credits within the same transaction, inserts a transaction and audit event, then commits. Any exception rolls back all updates.

Views, indexes, and triggers

OBJECT	PURPOSE
v_account_summary	Joins customer/account data with pre-aggregated transaction counts and unresolved alert counts. Pre-aggregation prevents totals from being multiplied by join fan-out.
v_open_alerts	Provides a reusable query of open/investigating alerts with transaction, account, and customer context.
Indexes	Support account/time history, transaction date, risk score, status, alert status/severity, alert account, customer/account joins, and audit timestamp retrieval.
trg_transaction_audit	Automatically inserts a TRANSACTION_CREATED audit record when a transaction row is inserted.
trg_alert_audit	Automatically inserts an ALERT_CREATED audit record when an alert is inserted.
trg_prevent_negative_balance	Aborts an account balance update if the resulting balance would be negative, complementing the table CHECK constraint.

05 / BUSINESS LOGIC

Fraud scoring & API workflows

Interpretable rules make the demo explainable during evaluation.

Rule-based transaction scoring

RULE	CONDITION	POINTS ADDED
High value	Amount is at least ₹100,000.	+65
Large ticket	Otherwise, amount is at least ₹50,000.	+42
Cross-border	International transaction is at least ₹10,000.	+35
Velocity	At least four same-account transactions fall in the 20-minute surrounding window.	+25
ATM pattern	ATM transaction is at least ₹25,000.	+20

The score is capped at 100 and never lowered by a scan. New alert rows are created at scores of 55 or above. Severity is Medium (55–69), High (70–89), or Critical (90–100). Existing alerts are not duplicated because transaction_id is unique in fraud_alerts. A live scan stores changed scores and writes a SCAN_COMPLETED audit entry.

Key HTTP API endpoints

METHOD / PATH	FUNCTION
GET /api/health	Check application and SQLite availability.
GET /api/dashboard	Metrics, six-month trend, categories, risk mix, and recent activity.
GET /api/transactions	Parameterized search with risk/status filters.
GET /api/alerts	Filter by status, severity, and search phrase.
GET /api/accounts	Account/customer summary built from v_account_summary.
GET /api/reports and /api/audit	Aggregate reports, query examples, and audit history.
GET /api/export/transactions	Export filtered transaction rows as CSV.
POST /api/scan	Apply scoring rules and create non-duplicate alerts.
POST /api/transfer	Atomic transfer; updates balances, transaction and audit rows together.
POST /api/alerts/{id}/decision	Change alert status/owner and record an audit event.

Simulated transfer workflow

1. Validate the amount, source/destination accounts, account status, and matching currency; begin with BEGIN IMMEDIATE before balance checks.
2. Conditionally debit the source, credit the destination, and insert transaction/audit rows. Commit together; on any failure, roll back every change.

06 / SQL EVIDENCE

Representative queries

The web application executes SQL; these examples can also be run in a SQLite tool.

Query 1 — customer/account transaction volume

```
SELECT c.full_name, a.id AS account_id,
    COUNT(t.id) AS transaction_count,
    ROUND(COALESCE(SUM(t.amount), 0), 2) AS gross_volume
FROM customers c
JOIN accounts a ON a.customer_id = c.id
LEFT JOIN transactions t ON t.account_id = a.id
GROUP BY c.id, a.id
ORDER BY gross_volume DESC;
```

Query 2 — high-risk transaction search

```
SELECT external_ref, account_id, merchant, amount, risk_score, created_at
FROM transactions
WHERE risk_score >= 70
ORDER BY risk_score DESC, created_at DESC;
```

Query 3 — alert outcomes

```
SELECT status, COUNT(*) AS alert_count
FROM fraud_alerts
GROUP BY status
ORDER BY alert_count DESC;
```

Query 4 — accounts with active alerts (view)

```
SELECT account_id, customer_name, balance, active_alert_count
FROM v_account_summary
WHERE active_alert_count > 0
ORDER BY active_alert_count DESC;
```

Query 5 — inspect query execution strategy

```
EXPLAIN QUERY PLAN
SELECT external_ref, amount, risk_score
FROM transactions
WHERE risk_score >= 70
ORDER BY risk_score DESC;
```

Example aggregate output

The preloaded dataset contains 94 transactions and 15 seeded alerts. At initialization, 10 alerts are active (Open or Investigating), seven are High/Critical, three are Resolved, and two are marked False Positive. Dashboard totals are recalculated from the current database, so these numbers may change after scans or analyst actions.

07 / VERIFICATION

Testing & execution

Smoke tests use an isolated temporary database so the bundled demo file is not changed.

TEST	ACTION	EXPECTED / OBSERVED RESULT
Static UI	GET / and request dashboard resources.	HTML interface is served from the local Python HTTP server.
Seed validation	Read the initial dashboard and account list.	Seeded customers/accounts/transactions and alert metrics are present.
Rule scan	POST /api/scan.	Transaction rules run; risk scores update and only new qualifying alerts are inserted.
Alert workflow	Change an alert to Investigating.	Status update succeeds and an audit event is recorded.
Valid transfer	Transfer a small amount between active accounts.	Debit, credit, transaction record, and audit entry commit together.
Insufficient funds	Attempt a transfer above the available balance.	API rejects the request; transaction rolls back with no partial transfer.
CSV export	GET /api/export/transactions.	CSV content type and header row are returned.
Reports endpoint	GET /api/reports.	Category, channel, alert disposition, and SQL example data are returned.
User-entered records	Create customer/account and a high-value international transaction, then scan.	New linked records persist; duplicate email is rejected; qualifying transaction generates an alert and audit events.

Windows setup

3. Extract the project ZIP to a writable folder.
4. Install Python 3.10+ and enable “Add Python to PATH” if Python is not already available.
5. Double-click run_phantom.bat and keep the terminal open.
6. Open http://127.0.0.1:8080 in Chrome or Edge.
7. Use Run threat scan, review the alert queue, change an alert outcome, inspect the audit trail, and try a small simulated transfer.
8. Stop the server with Ctrl+C in the terminal.

Validation note: the automated suite passed against an isolated SQLite database for the UI route, seeded dashboard, scan, alert decision, successful and rejected transfers, customer/account creation, manual transaction creation, duplicate-email rejection, scan-generated alert, audit events, CSV export, and reports.

Reset process

Stop the server, delete data/phantom.db and any phantom.db-wal or phantom.db-shm files, then launch the app again. The server recreates the schema and applies database/seed.sql automatically if the customer table is empty.

08 / EVALUATION

Limitations, future work & conclusion

The project emphasizes DBMS principles and an explainable demo rather than production banking compliance.

Current limitations

- No user authentication or role-based access control is implemented; the app is intended to remain on localhost for a classroom demo.
- The risk engine is a rules demonstration, not a calibrated fraud model. Thresholds can produce false positives and false negatives.
- Transaction data is fictional, and internal transfers are simulated. There is no bank, payment gateway, external IP intelligence, or live financial connection.
- SQLite is appropriate for a local single-user demo, but a production multi-user deployment would need a hardened API, access controls, deployment monitoring, backup strategy, and likely a server DBMS.
- Record creation is supported through validated forms, but full edit/delete workflows and bulk CSV import are future improvements.

Future enhancements

- Add authenticated analyst and administrator roles with least-privilege permissions.
- Add controlled customer/transaction editing, CSV import with preview/validation, and duplicate detection for batch uploads.
- Add configurable fraud rules and an analyst feedback loop to measure false-positive rates.
- Use PostgreSQL for a multi-user environment, database migrations, and controlled operational backups.
- Add richer device/IP history, alert case notes, evidence attachments, and an explainability panel for each individual rule.
- Add formal test coverage, versioned schema migrations, and accessibility/usability evaluation.

Conclusion

PHANTOM demonstrates how a real relational database can power a coherent software product. The user interface is connected to live SQL-backed endpoints; operators can create linked customer/account records and add transaction records; alerts reference stored transaction records; integrity constraints restrict invalid states; triggers preserve important audit events; and a transfer workflow showcases atomic updates and rollback. The deliverable includes the application source, reproducible seed data, database schema, ER model, testing script, setup instructions, and viva material so it can be run and explained locally.

Deliverable checklist

- ☑ Runnable frontend and backend
- ☑ SQLite file plus schema and seed SQL
- ☑ ER diagram in SVG and Mermaid source
- ☑ Automated smoke test
- ☑ Windows launcher and README
- ☑ Project report and viva preparation

END OF REPORT